



拷贝与移动

▼ 本质与区别

拷贝和移动的使用则取决于对象为左值还是右值，编译器会根据传递给构造函数的对象是左值（左值引用）还是右值（右值引用），来选择使用拷贝还是移动：

- **拷贝**：对左值进行操作，资源复制，可能导致性能下降。
- **移动**：对右值进行操作，资源转移，效率更高，避免了重复的内存分配和释放。

具体的实现过程为：

- **拷贝**：当进行拷贝时，创建了一个新的对象，该对象的值会被复制到新的内存位置。此时源对象和目标对象都有自己的独立副本。
- **移动**：当进行移动时，并不会复制数据，而是将源对象的资源所有权转移给目标对象。源对象通常处于有效但未定义的状态，但不再拥有原来的资源。这使得移动比拷贝更高效，因为它避免了资源的重复分配。

拷贝与移动本质上都是通过调用函数实现，拷贝通过拷贝构造函数和拷贝赋值运算符实现，移动则通过移动构造函数和移动赋值运算符实现。

▼ 构造与赋值

`A obj2 = obj1;` 与 `A obj2; obj2 = obj1;` 在效果上类似，但实际上是调用了不同的函数：

前者会调用对应类型的构造函数（`obj1` 为左值，则使用拷贝构造函数），后者则是使用对应类型的 `operator=` 函数。

▼ 深拷贝与浅拷贝

编译器对于基本数据类型（如 `int`、`char`、`float` 等），拷贝操作会直接将值复制到另一个变量。每个对象都拥有自己独立的值，并且没有共享内存的问题，这种拷贝叫做深拷贝。

而对于指针类型，则只是将一个指针的值（即内存地址）赋给另一个指针。这并没有复制数据本身，而是让两个不同的指针指向相同的地址，这种拷贝叫做浅拷贝。

对于自定义或第三方的类，则通过拷贝构造函数和拷贝赋值运算符实现，默认的拷贝构造函数和拷贝赋值运算符会将类中的各个成员对应拷贝，各个成员是深拷贝还是浅拷贝则取决于自身的类型。

如果想对指针类型实现深拷贝，则需要为指针分配一块新的内存：`int a1 = new int(a2);` 这种操作会分配新的内存并复制数据。

▼ 拷贝构造函数

在对左值进行拷贝时，本质上是使用对象左值调用构造函数：

```
▼ 拷贝构造函数 C++  
  
class A {  
public:  
    A(const A& other) { // 拷贝构造函数  
        // 进行深拷贝操作  
        data = other.data;  
    }  
    int data;  
};
```

如果存在自己定义的构造函数，则会调用该函数；

如果没有定义，则会将所有对应的类型成员进行拷贝。

所以在效果上，`A obj2 = obj1;` 等效于 `A obj2(obj1);`，不过在具体的实现上，编译器可能实现了不同的优化。

▼ 拷贝赋值运算符

在对左值进行赋值时，本质上是使用对象左值调用 `operator=` 函数：

```
▼ operator= C++  
  
class A {  
public:  
    int data;  
  
    A(int value) : data(value) {}  
  
    A& operator=(const A& other) {  
        if (this == &other) {  
            return *this; // 处理自赋值的情况  
        }  
        data = other.data; // 拷贝值  
        return *this;  
    }  
};
```

最后要返回自身的引用 `*this`，这样才可以支持链式赋值。

如果没有返回值，赋值链中的下一步将没有接受的参数。

▼ 移动构造函数

对于基本类型（如 `int`、`float` 等），在移动构造函数中，并不需要像指针那样管理内存，因为它们的内存通常是栈上的，自动管理。

因此，对于基本类型的成员变量，移动构造函数的实现通常只是进行**直接拷贝**，不需要做特殊的处理。

对于指针类型（如 `int*`、`std::string*` 等），在移动构造函数中，我们需要将源对象的指针转移给新对象，并将源对象的指针置为空。这样做是为了防止源对象在析构时删除指针指向的内存。

```
▼ 移动构造函数 C++
class A {
public:
    int value;           // 基本类型
    int* data;           // 指针类型

    // 构造函数
    A(int value, int data_value) : value(value), data(new int(data_value)) {}

    // 析构函数
    ~A() {
        delete data; // 释放动态分配的内存
    }

    // 移动构造函数
    A(A&& other) noexcept
        : value(other.value), data(other.data) { // 转移资源
        other.data = nullptr; // 将源对象的数据指针置空
    }
};
```

唯一需要注意的是，只有接受右值引用时，才会触发移动构造函数，所以在移动时，需要通过 `std::move` 将对象转换为右值引用。

移动构造函数和移动赋值运算符常用 `noexcept`，在实现移动构造函数和移动赋值运算符时，通常会将它们标记为 `noexcept`，因为移动操作不应该抛出异常。

尤其是对高效的容器实现来说，移动操作必须被声明为 `noexcept`。

▼ 移动赋值运算符

在使用 `=` 接受右值的赋值时，会触发移动赋值运算符。

```
▼ operator= C++
class A {
public:
    int value;
    int* data; // 动态分配内存

    A(int value, int data_value) : value(value), data(new int(data_value)) {}

    ~A() {
```

```

        delete data; // 析构函数释放内存
    }

    // 移动赋值运算符
    A& operator=(A&& other) noexcept {
        if (this != &other) { // 防止自赋值
            delete data; // 释放当前对象的数据
            value = other.value;
            data = other.data;
            other.data = nullptr; // 将源对象的指针置空
        }
        return *this;
    }
}

```

▼ 显式禁用函数

在一些情况，可能需要禁止拷贝与赋值，此时可以使用显式禁用函数语法：

```

A(const A&) = delete;
A& operator=(const A&) = delete;

```

C++

通过这种方法，可以禁用对应类型的构造函数和赋值运算符。